

PMS & External System Integration Architecture

Technical Implementation Guide for Tourism Post-Stay Platform

Document Type: Integration Architecture Blueprint

Author: Platform Engineering Team

Date: February 10, 2026

Target Audience: Integration developers, PMS vendors, DevOps teams

Classification: Technical Documentation

Executive Summary

This document specifies the connector architecture for integrating Property Management Systems (PMS), channel managers, and third-party review platforms into the Tourism Post-Stay Intelligence Platform. The architecture follows an **adapter pattern** with thin, stateless connectors that normalize vendor-specific APIs into platform-internal events, ensuring privacy-first data handling, fault tolerance, and scalability across multiple properties[1][21].

Key objectives:

- Establish vendor-agnostic integration layer supporting multiple PMS/channel manager systems simultaneously
 - Enforce privacy boundaries: strip PII before events enter AI/analytics pipeline
 - Enable real-time synchronization for checkout events, bookings, and review ingestion
 - Provide fallback mechanisms for legacy systems without webhook support
 - Maintain complete audit trail for all external data flows
-

1. Integration Overview

1.1 Target Integration Categories

The platform integrates with three primary external system categories:

Category	Purpose	Examples
PMS Systems	Guest stay lifecycle, checkout events, booking metadata	Slope, 5 Stelle, Passepartout, AKAI Smart PMS
Channel Managers	OTA bookings, availability sync, rate distribution	Booking Expert, Cbooking, Room In Cloud, Wubook
Review Platforms	Public review ingestion, rating data	Google Reviews, Booking.com, TripAdvisor

Table 1: External system integration categories

From the provided screenshot, initial MVP will prioritize Italian hospitality market PMS vendors (Slope, 5 Stelle, Passepartout, AKAI Smart PMS) and channel managers (Booking Expert, Cbooking, Room In Cloud, Wubook).

1.2 Integration Patterns

Pattern	Method	Latency	Use Case
Webhook (preferred)	Real-time push events	<5 seconds	Modern PMS with event API
Scheduled polling	Periodic API sync	5-30 minutes	Legacy PMS with read-only API
Batch file transfer	SFTP/FTP CSV drop	Daily	No-API legacy systems
Manual entry	Staff-initiated via console	On-demand	Exceptional cases, data correction

Table 2: Integration pattern hierarchy by preference

Pattern selection criteria:

- Webhook if vendor supports POST callback API with authentication
 - Scheduled polling if vendor exposes REST/SOAP API with OAuth2 or API key
 - Batch transfer for vendors without API (CSV export capability required)
 - Manual entry reserved for incident response or small-volume properties
-

2. Adapter Architecture

2.1 Design Principles

1. **Thin, Stateless Adapters:** Each connector is a lightweight translation layer with no business logic. State maintained in platform core, not adapters.
2. **Normalize First:** Vendor-specific payloads transformed into platform-internal event schema immediately upon ingestion, before any storage or processing[21][24].
3. **Privacy Boundaries Enforced at Adapter:** PII fields stripped or pseudonymized before event enters message queue. Adapter layer is first enforcement point[1].
4. **Idempotency by Design:** All inbound events assigned unique event_id (hash of vendor booking_id + timestamp). Duplicate webhooks/polls detected and deduplicated[21][26].
5. **Version Independence:** Multiple adapter versions for same vendor can coexist. Properties configure which version to use, enabling gradual migration when vendor changes API[23][25].
6. **Fail-Safe Defaults:** Connection failures, timeouts, or validation errors trigger alerts but never block property operations. Fallback to manual workflow if automated sync unavailable[21].

2.2 Adapter Lifecycle

Figure 1: Adapter request/response flow

1. **Ingestion:** External system sends data via webhook POST, platform polls API endpoint, or CSV file arrives via SFTP

2. **Authentication:** Verify webhook signature (HMAC), validate OAuth2 token, or check API key against secrets vault[25][28]
3. **Schema Validation:** Validate payload against vendor-specific JSON schema; reject malformed requests with HTTP 400 + log incident
4. **Normalization:** Map vendor fields to platform-internal event schema (see Section 3)
5. **PII Pseudonymization:** Generate `guest_key` from vendor `booking_id/guest_id` using HMAC; store mapping in encrypted Identity Vault; strip name, email, phone from event payload[1]
6. **Event Emission:** Publish normalized, pseudonymized event to internal message queue (Redis/RabbitMQ) with `event_type` tag
7. **Acknowledgment:** Return HTTP 200 (webhook) or update sync cursor (polling) to prevent re-processing
8. **Audit Logging:** Record adapter transaction: `vendor_id`, `event_id`, timestamp, success/failure, payload hash

2.3 Adapter Components

Each adapter implemented as standalone microservice or serverless function with:

- **Webhook receiver endpoint** (Express.js/FastAPI route): POST `/webhooks/:vendor/:event_type`
 - **Polling scheduler** (cron job or Lambda scheduled event): runs every N minutes
 - **Schema validator** (JSON Schema or Pydantic models): vendor-specific request/response validation
 - **Field mapper** (transformation logic): `vendor_field` → `platform_field` dictionary
 - **Secrets client** (AWS Secrets Manager/Vault): fetch API keys, HMAC secrets
 - **Event publisher** (Redis client/SQS SDK): push to internal queue
 - **Metrics emitter** (StatsD/CloudWatch): track success rate, latency, error types
-

3. Platform-Internal Event Schema

3.1 Core Event Types

All adapters emit events conforming to these standardized types:

Event Type	Trigger	Downstream Action
GuestCheckedOut	Checkout timestamp recorded in PMS	Survey dispatch workflow
BookingCreated	New reservation confirmed	Pre-arrival communication (future)
BookingModified	Dates, room type, guest count changed	Update CRM timeline
BookingCancelled	Reservation cancelled by guest/property	Stop survey, log in analytics
ReviewPublished	New review on Google/Booking/TripAdvisor	AI analysis + response draft
PaymentProcessed	Payment settled (for upsell tracking)	Revenue attribution (future)

Table 3: Platform-internal event taxonomy

3.2 Event Schema Structure

All events follow this JSON structure:

```
{
  "event_id": "evt_a1b2c3d4e5f6",
  "event_type": "GuestCheckedOut",
  "event_version": "1.0",
  "timestamp": "2026-02-10T19:30:00Z",
  "source": {
    "vendor": "slope_pms",
    "adapter_version": "2.1.0",
    "property_id": "prop_12345"
```

```
},
"payload": {
  "guest_key": "gk_9f8e7d6c5b4a",
  "stay_id": "stay_abc123",
  "booking_id": "slope_res_7890",
  "checkout_ts": "2026-02-10T11:00:00Z",
  "checkin_ts": "2026-02-08T15:00:00Z",
  "room_type": "deluxe_double",
  "booking_channel": "direct",
  "guest_language": "it",
  "guest_country": "IT",
  "consent_marketing": true,
  "consent_ai_analysis": true
},
"metadata": {
  "idempotency_key": "slope_7890_20260210110000",
  "processed_at": "2026-02-10T19:30:05Z"
}
}
```

Critical fields:

- **guest_key**: Pseudonymous identifier, never raw name/email
- **stay_id**: Platform-generated unique ID for this stay instance
- **booking_id**: Vendor's original reservation ID (for troubleshooting only)
- **consent_***: Boolean flags from PMS if available, else default to false

3.3 Field Mapping Examples

Slope PMS Field	Platform Field	Transform
id_prenotazione	booking_id	Prepend "slope_res_"
cognome_ospite	guest_key	HMAC(cognome_ospite + nome_ospite, secret)
data_partenza	checkout_ts	Parse DD/MM/YYYY → ISO 8601 UTC
tipo_camera	room_type	Normalize to taxonomy (standard, superior, deluxe, suite)
canale_vendita	booking_channel	Map: "diretto" → "direct", "booking" → "ota_booking"
lingua_preferenza	guest_language	ISO 639-1 code (it, en, de, fr)

Table 4: Example field mapping for Slope PMS adapter

Similar mapping tables created for each vendor. Stored in version-controlled configuration files (YAML/JSON).

4. PMS-Specific Integration Specifications

4.1 Webhook-Based PMS (Modern Systems)

Supported vendors: AKAI Smart PMS, Mews PMS (future), Cloudbeds (future)

Flow:

1. Property configures webhook URL in PMS admin panel:
https://platform.example.com/webhooks/akai/checkout
2. PMS sends POST request on guest checkout with JSON payload
3. Platform webhook endpoint validates signature (HMAC-SHA256 using shared secret)
4. Adapter normalizes payload, emits GuestCheckedOut event
5. Returns HTTP 200 acknowledgment within 3 seconds (PMS retry timeout)

Webhook authentication:

- Shared secret provisioned during property onboarding
- PMS includes X-Signature header: HMAC-SHA256(request_body, secret)
- Platform recomputes signature, rejects if mismatch (HTTP 401)

Retry logic:

- If webhook endpoint down, PMS retries with exponential backoff (vendor-dependent)
- Platform deduplicates via idempotency_key (hash of booking_id + timestamp)

Error handling:

- Malformed JSON → HTTP 400 + alert to ops team
- Missing required fields → HTTP 422 + log for vendor support escalation
- Signature mismatch → HTTP 401 + security incident log

4.2 Polling-Based PMS (REST API)

Supported vendors: Slope, 5 Stelle, Passepartout

Flow:

1. Platform scheduler triggers every 30 minutes
2. Adapter queries PMS API: GET /api/checkouts?since=<last_sync_timestamp>
3. Authenticate with OAuth2 access token (refresh if expired) or API key
4. Parse response JSON array of checkout records
5. For each record: normalize, pseudonymize, emit event
6. Update sync cursor in Redis: SET adapter:slope:last_sync <current_timestamp>

OAuth2 implementation (if supported):

- Client credentials flow: exchange client_id + client_secret for access_token
- Token cached in memory with TTL (typically 3600s)
- Refresh token used if access_token expires mid-sync

API key implementation (fallback):

- Static API key stored in secrets vault
- Included in request header: Authorization: Bearer <api_key>

Pagination handling:

- If API returns >100 records, follow next_page links
- Maximum 10 pages per sync cycle (prevents runaway queries)

Rate limiting:

- Respect vendor rate limits (e.g., 60 requests/minute for Slope)
- Implement token bucket or leaky bucket algorithm in adapter
- Back off if receiving HTTP 429 responses

4.3 Batch File Transfer (Legacy PMS)

Supported vendors: Older versions of Passepartout, regional PMS without API

Flow:

1. Property exports daily checkout report from PMS as CSV
2. File uploaded via SFTP to platform drop zone:
/incoming/property_12345/checkouts_20260210.csv
3. Platform file watcher detects new file (inotify/S3 event)
4. CSV parser validates structure (expected columns: booking_id, guest_surname, checkout_date, room_type)
5. For each row: normalize, pseudonymize, emit event
6. Move processed file to archive:
/processed/property_12345/checkouts_20260210.csv
7. Send confirmation email to property admin

CSV schema validation:

- Required columns defined in per-vendor config
- Type checking: dates in DD/MM/YYYY or ISO format, numeric fields for amounts
- Reject file if >10% rows fail validation

Security:

- SFTP with key-based authentication (no passwords)
- Platform generates SSH key pair per property, public key provided during onboarding
- Files encrypted at rest in S3 with KMS

Reconciliation:

- Daily report: X records processed, Y skipped (duplicates), Z errors
 - Property admin can download error log from console
-

5. Channel Manager Integration

5.1 Purpose and Scope

Channel managers (Booking Expert, Cbooking, Room In Cloud, Wubook) distribute inventory and rates to OTAs. Platform integrates to capture:

- Bookings originating from OTAs ([Booking.com](#), Expedia, Airbnb)
- Booking channel attribution (which OTA generated the reservation)
- Guest contact info for survey dispatch (if permitted by OTA terms)

Integration pattern: Primarily webhook-based, as channel managers aggregate multiple OTA feeds and emit unified events[24][26].

5.2 Channel Manager Webhook Schema

Channel managers typically send BookingCreated event with structure:

```
{
  "event": "new_booking",
  "booking_id": "wubook_12345",
  "property_code": "HOTEL_ABC",
  "checkin": "2026-03-15",
  "checkout": "2026-03-18",
  "guest": {
```

```
"first_name": "Mario",
"last_name": "Rossi",
"email": "mario.rossi@example.com",
"phone": "+39 123 4567890"
},
"room_type": "standard_double",
"channel": "booking_com",
"total_price": 450.00,
"currency": "EUR"
}
```

Adapter normalization:

- Generate guest_key from hash of last_name + email
- Store PII in Identity Vault (encrypted)
- Emit BookingCreated event with guest_key, no PII in payload
- Tag with booking_channel: "ota_booking"

Privacy consideration: Some OTAs (e.g., [Booking.com](#)) obscure guest email until after check-in. Adapter handles missing email gracefully, using booking_id as pseudonymization seed if necessary.

5.3 Two-Way Sync (Future Extension)

Future phases may implement outbound sync to channel managers:

- Send availability/rate updates from platform to OTAs (requires PMS write-back)
- Close-out dates automatically based on property calendar
- Dynamic pricing adjustments from revenue management module

This requires reverse adapters (platform → channel manager API) with similar authentication and retry patterns[27][30].

6. Review Platform Integration

6.1 Supported Platforms

Platform	API Type	Auth Method
Google My Business	REST API	OAuth2 (service account)
Booking.com	Partner Hub API	OAuth2 (client credentials)
TripAdvisor	Content API	API key

Table 5: Review platform integration methods

6.2 Review Ingestion Flow

1. Daily cron job (2 AM UTC) triggers review sync for all connected properties
2. For each property + platform:
 - Query API for new reviews: GET /reviews? location_id=X&since=<last_sync>
 - Parse response: review_id, rating, text, author_name, published_date, language
 - Attempt to match review to stay_id via heuristics (date range +/- 7 days, guest_key if author email known)
 - Emit ReviewPublished event with review text + metadata
3. AI analysis triggered on event receipt (sentiment, themes, draft response)
4. If review negative (rating ≤ 3) or contains urgent keywords, alert guest relations team

Matching logic:

- If review published 2026-02-12 and property has checkout on 2026-02-10, likely match (± 2 day tolerance)
- If reviewer name/email matches guest_key in Identity Vault, definitive match
- Otherwise store as anonymous property feedback (no guest_key association)

Rate limiting:

- Google: 100 requests/property/day (batch fetch)

- [Booking.com](#): 1000 requests/day across all properties
- TripAdvisor: 500 requests/day

Error handling:

- API quota exceeded → skip sync, retry next day
 - Authentication failure → alert admin, require re-authorization
-

7. Security and Privacy Controls

7.1 Privacy Boundaries in Adapters

Adapter layer is first privacy checkpoint:

1. **PII Stripping**: Name, email, phone, passport/ID never included in normalized event payload
2. **Pseudonymization**: Generate `guest_key` = HMAC-SHA256(`booking_id` + `property_secret`) immediately upon ingestion
3. **Encrypted Storage**: Mapping `guest_key` ↔ PII stored in separate Identity Vault table with field-level encryption (AES-256)
4. **Minimal Context**: Event payload includes only `guest_key`, stay attributes (`room_type`, `dates`, `channel`), `language`, `country`—no free-text notes or special requests from PMS

Vault access control:

- Only backend services with explicit IAM role can decrypt Identity Vault entries
- Staff console cannot query vault directly; requests must go through privacy-controlled API layer
- All vault access logged with `user_id`, `purpose`, `timestamp`

7.2 Authentication and Authorization

Integration Direction	Auth Mechanism
Inbound webhooks (PMS → Platform)	HMAC signature validation
Outbound API calls (Platform → PMS)	OAuth2 client credentials or API key
SFTP file transfer	SSH key pairs (no password)
Review platform APIs	OAuth2 service account or API key

Table 6: Authentication methods by integration type

Secret rotation:

- HMAC secrets rotated every 90 days via automated vault workflow
- API keys/tokens refreshed when vendor requires or on security incident
- SSH keys rotated annually; old keys revoked immediately after replacement deployed

7.3 Audit Logging

Every adapter transaction logged with:

- adapter_id: Which vendor adapter processed event
- event_id: Unique event identifier
- timestamp: UTC timestamp of ingestion
- source_ip: Webhook sender IP (if applicable)
- payload_hash: SHA-256 hash of raw payload (for forensics, not stored verbatim)
- outcome: success | validation_error | auth_failure | processing_error
- latency_ms: Time from receipt to event emission

Logs retained 90 days in hot storage (CloudWatch/ELK), then archived to cold storage for 5 years (compliance).

8. Fault Tolerance and Reliability

8.1 Retry Strategies

Failure Type	Retry Strategy	Fallback
Webhook endpoint unavailable	Rely on vendor retry (3-5 attempts)	Alert ops team, manual CSV import
PMS API timeout (polling)	Exponential backoff: 30s, 60s, 120s	Skip sync cycle, resume next scheduled run
Invalid JSON schema	No retry, log error	Alert property admin to fix PMS config
Identity Vault unavailable	Retry 3× with 5s delay	Queue event for delayed processing
Message queue full	Back-pressure: return HTTP 503 to PMS	Vendor retries later

Table 7: Fault tolerance matrix

8.2 Idempotency Guarantees

Deduplication mechanism:

1. Compute `idempotency_key = SHA-256(vendor + booking_id + checkout_ts)`
2. Check Redis: `EXISTS processed_events:<idempotency_key>`
3. If exists (TTL 7 days): discard duplicate, log, return HTTP 200
4. If new: process event, set `SET processed_events:<idempotency_key> 1 EX 604800`

Why 7-day TTL: Covers typical vendor retry windows (24-72 hours) with safety margin.

8.3 Circuit Breaker Pattern

For polling adapters:

- Track consecutive failures per vendor (Redis counter)
- If 5 consecutive failures → open circuit (stop polling for 1 hour)
- After 1 hour → half-open state (attempt 1 sync)
- If success → close circuit (resume normal schedule)
- If failure → reopen circuit (extend backoff to 4 hours)

Prevents runaway retry storms when vendor API has prolonged outage.

8.4 Monitoring and Alerting

Metrics tracked per adapter:

- `events_received_total`: Counter of all inbound events
- `events_processed_success`: Counter of successfully normalized events
- `events_failed_validation`: Counter of schema validation failures
- `adapter_latency_ms`: Histogram of processing time (p50, p95, p99)
- `auth_failures_total`: Counter of authentication rejections

Alert thresholds:

- Validation failure rate >10% over 1 hour → notify ops team
- No events received in 24 hours for active property → check PMS connectivity
- Auth failure spike (>5 in 15 min) → potential credential compromise, rotate secrets
- Latency p95 >5 seconds → investigate queue backlog or vault performance

9. Property Configuration and Onboarding

9.1 Per-Property Connector Settings

Each property configures which integrations are active via admin console:

Setting	Options
PMS Vendor	Slope 5 Stelle Passepartout AKAI None (manual)
Integration Method	Webhook Polling SFTP Manual
Sync Schedule (polling)	Every 30 min Every 1 hour Every 6 hours
Channel Manager	Booking Expert Cbooking Room In Cloud Wubook None
Review Platforms	Google Booking.com TripAdvisor (multi-select)
Historical Data Import	Yes (import last 90 days) No (only new data)

Table 8: Property-level integration configuration

9.2 Onboarding Workflow

- 1. Property Admin Initiates:** Select PMS vendor from dropdown in console
- 2. Platform Generates Credentials:**
 - For webhooks: generate HMAC secret, display webhook URL to copy
 - For polling: display instructions to obtain OAuth2 client ID/secret or API key from PMS admin panel
 - For SFTP: generate SSH key pair, provide public key for property to upload to PMS
- 3. Property Configures PMS:**
 - Enter webhook URL in PMS settings
 - OR: Authorize platform OAuth2 app in PMS admin
 - OR: Configure SFTP server details
- 4. Platform Tests Connection:**
 - Webhook: send test event from PMS to verify receipt
 - Polling: attempt API call, verify authentication success

- SFTP: test upload/download with dummy file
5. **Historical Import (optional):**
- If enabled, trigger one-time batch import of last 90 days checkouts
 - Process offline to avoid rate limits
 - Display progress bar in console
6. **Go Live:** Integration status changes to "Active" in console; events begin flowing

Estimated onboarding time:

- Webhook PMS: 15-30 minutes (configure + test)
- Polling PMS: 30-60 minutes (OAuth setup + test)
- SFTP: 1-2 hours (coordinate with IT, SSH keys)

9.3 Per-Integration Toggle Controls

Property can enable/disable individual integrations without full disconnection:

- **Pause PMS sync:** Stop processing new checkout events (e.g., during PMS migration)
- **Pause review ingestion:** Stop fetching reviews from specific platform (e.g., dispute with OTA)
- **Test mode:** Process events but don't trigger downstream workflows (survey dispatch disabled)

All toggles logged in audit trail with user_id and reason.

10. Adapter Versioning and Migration

10.1 Version Strategy

Each adapter tagged with semantic version: vendor_adapter-v2.1.0

- **Major version (2.x.x):** Breaking schema changes, requires property re-onboarding
- **Minor version (x.1.x):** New optional fields, backward-compatible
- **Patch version (x.x.1):** Bug fixes, no schema impact

Multi-version support:

- Platform can run `slope_adapter-v2.1.0` and `slope_adapter-v3.0.0` simultaneously
- Property settings specify which version to use
- Enables gradual migration: pilot properties on v3, others remain on v2

10.2 Migration Process

When vendor releases breaking API change (e.g., Slope PMS v5 API):

1. **Develop new adapter version:** Clone v2 adapter, update to v3 schema
2. **Deploy to staging:** Test with sandbox property using vendor's test API
3. **Pilot with 2-3 properties:** Invite early adopters to switch in console settings
4. **Monitor for 2 weeks:** Track error rates, latency, data quality vs. v2
5. **Gradual rollout:** Email all Slope properties with migration guide
6. **Sunset v2:** After 90 days, deprecate v2 (require all properties to migrate or disable integration)

Deprecation policy:

- 90-day notice before forced migration
- Email notifications at 90, 60, 30, 14, 7 days before cutoff
- Properties still on deprecated version at cutoff: integration auto-disabled, manual CSV workflow fallback

11. Development Standards

11.1 Adapter Code Structure

Each adapter repository organized as:

```
adapters/  
├── slope_pms/  
│   ├── v2/  
│   └── src/
```

```
| | | | — webhook.ts # Webhook handler
| | | | — poller.ts # Scheduled polling
| | | | — mapper.ts # Field normalization
| | | | — auth.ts # OAuth2/API key client
| | | | — schema.ts # JSON schema definitions
| | | — config/
| | |   | — field_mapping.yaml
| | | — tests/
| | |   | — webhook.test.ts
| | |   | — fixtures/ # Sample PMS payloads
| | | — Dockerfile
| | | — README.md
| — v3/
| — ...
| — cbooking/
|   | — v1/
|   | — ...
| — shared/
| — pseudonymization.ts # guest_key generation
| — event_publisher.ts # Redis/SQS client
| — secrets_client.ts # Vault integration
```

11.2 Testing Requirements

Unit tests:

- Test field mapping with real vendor payload fixtures
- Test authentication signature validation
- Test idempotency key generation

Integration tests:

- Mock vendor API/webhook, verify end-to-end event emission
- Test error scenarios: malformed JSON, auth failure, rate limit

Contract tests:

- Validate vendor API response schema hasn't changed (Pact or similar)
- Run nightly against vendor staging environments

Load tests:

- Simulate 1000 concurrent webhook POSTs (burst scenario: conference checkout)
- Verify latency <1s p99, no dropped events

11.3 Deployment Pipeline

1. Developer commits to adapters/slope_pms/v2 branch
2. GitHub Actions triggers:
 - Lint (ESLint/PyLint)
 - Unit tests (Jest/Pytest)
 - Build Docker image
 - Push to ECR/Docker Hub with tag slope_adapter:v2.1.0-sha-abc123
3. Deploy to staging ECS cluster
4. Run integration test suite against staging
5. Manual approval required for production deploy (Slack notification)
6. Deploy to production ECS (blue-green deployment)
7. Monitor error rate for 1 hour; auto-rollback if >1% failure rate

Rollback procedure:

- Revert ECS task definition to previous version
- Update DNS/load balancer to route to old tasks
- Post-incident review within 24 hours

12. Cost and Performance Optimization

12.1 API Call Budget

Monthly limits per property:

- PMS polling: 1440 calls/month (30 min intervals × 48/day × 30 days)
- Review platforms: 90 calls/month (daily sync × 3 platforms × 30 days)
- Channel manager: ~500 webhook deliveries/month (estimated booking volume)

Vendor API costs (if applicable):

- Most PMS/channel managers: free API access for integration partners
- Google My Business API: free (rate-limited)
- [Booking.com](#) Partner Hub: free with connectivity agreement

Platform costs:

- Adapter compute: ~0.5 vCPU per property (ECS Fargate spot instances)
- Message queue: <\$1/month per property (Redis/SQS)
- Secrets storage: <\$0.50/month per property (AWS Secrets Manager)

Total infrastructure cost: ~\$5-10/property/month for integration layer[21][26].

12.2 Latency Optimization

Target SLAs:

- Webhook processing: <1s p95 (receipt to event emission)
- Polling cycle: <30s p95 (API call to event emission for 100 checkouts)
- CSV batch: <5 min for 1000 rows

Optimization techniques:

- **Async event emission:** Webhook handler returns HTTP 200 immediately after validation, processes normalization in background worker
 - **Batch event publishing:** Accumulate 10 events, publish to queue in single transaction (reduces Redis RTT)
 - **Connection pooling:** Reuse HTTP connections to PMS APIs (keep-alive)
 - **Caching:** Cache OAuth2 tokens, field mapping configs in memory (TTL 1 hour)
-

13. Vendor-Specific Adapter Notes

13.1 Slope PMS

API Documentation: Assumed REST API with OAuth2 (verify with vendor)

Key fields:

- `id_prenotazione` → `booking_id`
- `data_partenza` → `checkout_ts` (format: DD/MM/YYYY HH:mm)
- `cognome_ospite` + `nome_ospite` → pseudonymize to `guest_key`

Polling endpoint: GET `/api/v1/checkouts?from=YYYY-MM-DD&to=YYYY-MM-DD`

Rate limit: 60 requests/minute

Known issues: Timestamp field sometimes missing seconds (default to 00)

13.2 5 Stelle

API Documentation: SOAP-based legacy API (check if REST available)

Key fields:

- `NumReservation` → `booking_id`
- `DataPartenza` → `checkout_ts` (ISO 8601)

Polling endpoint: SOAP action `GetCheckoutsAfterDate`

Authentication: Username + password in SOAP header

Known issues: No webhook support; polling only

13.3 Passepartout

API Documentation: Limited API; CSV export preferred

CSV columns: ReservationID, GuestSurname, CheckoutDate, RoomType, Channel

SFTP details: Property uploads to
/exports/checkouts_YYYYMMDD.csv daily at 3 AM

Known issues: Manual export required for some older versions

13.4 AKAI Smart PMS

API Documentation: Modern REST API with webhook support

Webhook events: checkout.completed, booking.created,
booking.cancelled

Webhook signature: HMAC-SHA256 in X-AKAI-Signature header

Polling endpoint (fallback): GET /api/v2/events?
type=checkout&since=<timestamp>

Rate limit: 120 requests/minute

Known issues: Webhook delivery can be delayed 5-10 minutes during
high load

13.5 Booking Expert (Channel Manager)

API Documentation: REST API + webhooks for OTA bookings

Webhook events: new_booking, booking_modified,
booking_cancelled

Key fields:

- booking_ref → booking_id
- source_channel → booking_channel (maps to OTA name)

Rate limit: 100 requests/minute

Known issues: Guest email sometimes masked as
guest_<random>@bookingexpert.com until check-in

14. Compliance and Data Governance

14.1 GDPR Considerations

Data minimization in adapters:

- Adapters never log full PII in plaintext (logs contain only guest_key)
- Raw PMS payloads not stored; only normalized events retained

Right to erasure:

- When guest requests deletion, guest_key removed from Identity Vault
- Historical events remain with guest_key=NULL (preserves analytics aggregates)

Data Processing Agreements:

- DPA signed with each PMS/channel manager vendor
- Clause specifying platform as processor, property as controller
- Sub-processor disclosure for AI vendors (OpenAI, etc.)

14.2 PCI-DSS (if handling payment data)

Current scope: Platform does NOT process payment card data (PMS handles payments)

Future extension: If upselling requires payment capture, integrate with PCI-compliant gateway (Stripe, Adyen), never store card numbers in platform

15. Future Roadmap

15.1 Phase 1 (Months 1-4): MVP Integration

- Webhook support for AKAI Smart PMS
- Polling support for Slope, 5 Stelle
- SFTP support for Passepartout
- Google Reviews API integration
- Support 1-2 properties per vendor for pilot

15.2 Phase 2 (Months 5-8): Scale and Reliability

- Add channel manager integrations: Booking Expert, Cbooking
- Implement circuit breaker and advanced retry logic
- Multi-property dashboard: integration health across portfolio
- Webhook replay tool for incident recovery

15.3 Phase 3 (Months 9-12): Advanced Features

- Two-way sync: push availability/rates to channel managers
- Real-time PMS write-back: auto-create guest profiles in PMS from direct bookings
- Predictive sync: pre-fetch data based on historical checkout patterns
- Open API for third-party integrations (CRM, BI tools)

16. Conclusion

This integration architecture provides a scalable, privacy-first foundation for connecting the Tourism Post-Stay Intelligence Platform with diverse PMS, channel manager, and review platform ecosystems. By enforcing adapter-level pseudonymization, supporting multiple integration patterns (webhook, polling, batch), and maintaining strict version control, the platform can reliably ingest external data while preserving GDPR compliance and operational fault tolerance[1][21][26].

The thin adapter pattern ensures vendor-specific complexity remains isolated, enabling rapid onboarding of new PMS systems without disrupting core platform logic. Combined with comprehensive monitoring, idempotency guarantees, and graceful degradation strategies, the integration layer serves as a robust bridge between fragmented hospitality software landscape and unified post-stay intelligence workflows.

References

- [1] AI4Business. (2026). PMI e intelligenza artificiale: il paradosso dell'uso diffuso senza sistema. <https://www.ai4business.it/intelligenza-artificiale/pmi-e-intelligenza-artificiale-il-paradosso-delluso-diffuso-senza-sistema/>
- [21] ExploreTech. (2025). PMS integrations explained: leading vendors and the hotel tech stack. <https://www.explorettech.io/en/blog/pms-integrations-explained-leading-vendors-and-the-hotel-tech-stack>
- [23] BookingWhizz. (2025). PMS Reservation Push API. <https://crmdocs.bookingwhizz.com/api-integration>
- [24] SaaSaro. (2025). How an Integrated Channel Manager API Connects PMS and OTAs in Real-Time. <https://saasaro.com/blog/how-a-n-integrated-channel-manager-api-connects-pms-and-otas-in-real-time/>
- [25] Oracle. (2022). Using the Oracle Hospitality Adapter with Oracle Integration Generation 2. <https://docs.oracle.com/en/cloud/paas/integration-cloud/hospitality-adapter/oracle-hospitality-adapter-capabilities.html>
- [26] LinkedIn. (2026). Hotel PMS Connectivity API: Complete Guide for Hotels in 2026. <https://www.linkedin.com/pulse/hotel-pms-connectivity-api-complete-guide-hotels-2026-anshul-bansal-5rf0c>
- [27] SaaSaro. (2025). Channel Manager API Explained: A Simple Guide for Hoteliers. <https://saasaro.com/blog/channel-manager-api-explained-a-simple-guide-for-hoteliers/>
- [28] Oracle. (2022). Oracle Hospitality Adapter Capabilities. <https://docs.oracle.com/en/cloud/paas/application-integration/hospitality-adapter/oracle-hospitality-adapter-capabilities.html>
- [30] LinkedIn. (2026). How Channel Manager API Connectivity Transforms Hotel Operations. <https://www.linkedin.com/pulse/how-channel-manager-api-connectivity-transforms-hotel-operations-zirlc>
-

Appendix A: Adapter API Contract Examples

A.1 Webhook Request (AKAI PMS → Platform)

Endpoint: POST /webhooks/akai/checkout

Headers:

Content-Type: application/json

X-AKAI-Signature: sha256=a1b2c3d4e5f6...

X-AKAI-Event: checkout.completed

Body:

```
{
  "event_id": "akai_evt_12345",
  "timestamp": "2026-02-10T11:00:00Z",
  "property_code": "HOTEL_ABC",
  "reservation": {
    "id": "RES-7890",
    "guest": {
      "first_name": "Mario",
      "last_name": "Rossi",
      "email": "mario.rossi@example.com",
      "phone": "+39 123 456 7890"
    },
    "checkin_date": "2026-02-08",
    "checkout_date": "2026-02-10",
    "room_type": "deluxe_double",
    "booking_channel": "direct",
    "language": "it",
    "country": "IT"
  }
}
```

Platform Response:

HTTP/1.1 200 OK

Content-Type: application/json

```
{
  "status": "received",
```

```
"event_id": "evt_platform_abc123"  
}
```

A.2 Polling Request (Platform → Slope PMS)

Endpoint: GET /api/v1/checkouts?from=2026-02-10&to=2026-02-10

Headers:

Authorization: Bearer <oauth2_access_token>

Accept: application/json

Response:

```
{  
  "checkouts": [  
    {  
      "id_prenotazione": "SLOPE-001",  
      "cognome_ospite": "Rossi",  
      "nome_ospite": "Mario",  
      "email_ospite": "mario.rossi@example.com",  
      "data_arrivo": "08/02/2026 15:00",  
      "data_partenza": "10/02/2026 11:00",  
      "tipo_camera": "Superior",  
      "canale_vendita": "diretto",  
      "lingua_preferenza": "it"  
    }  
  ],  
  "total": 1,  
  "page": 1  
}
```

A.3 Platform-Internal Event (Normalized)

Queue: events.checkout

Message:

```
{  
  "event_id": "evt_a1b2c3d4e5f6",  
  "event_type": "GuestCheckedOut",  
  "event_version": "1.0",  
  "timestamp": "2026-02-10T19:30:00Z",  
  "source": {
```

```
"vendor": "slope_pms",
"adapter_version": "2.1.0",
"property_id": "prop_12345"
},
"payload": {
"guest_key": "gk_9f8e7d6c5b4a",
"stay_id": "stay_abc123",
"booking_id": "slope_SLOPE-001",
"checkout_ts": "2026-02-10T11:00:00Z",
"checkin_ts": "2026-02-08T15:00:00Z",
"room_type": "superior",
"booking_channel": "direct",
"guest_language": "it",
"guest_country": "IT",
"consent_marketing": true,
"consent_ai_analysis": true
},
"metadata": {
"idempotency_key": "slope_SLOPE-001_20260210110000",
"processed_at": "2026-02-10T19:30:05Z"
}
}
```

Appendix B: Field Mapping Configuration (YAML Example)

File: adapters/slope_pms/v2/config/field_mapping.yaml

```
vendor: slope_pms
version: 2.1.0
```

```
field_mappings:
  booking_id:
    source: id_prenotazione
    transform: prepend_vendor # slope_<value>
```

```
  guest_key:
    source: [cognome_ospite, nome_ospite, email_ospite]
    transform: pseudonymize # HMAC of concatenated fields
```

checkout_ts:
source: data_partenza
transform: parse_datetime # DD/MM/YYYY HH:mm → ISO 8601 UTC

checkin_ts:
source: data_arrivo
transform: parse_datetime

room_type:
source: tipo_camera
transform: normalize_room_type
mapping:
Standard: standard
Superior: superior
Deluxe: deluxe
Suite: suite

booking_channel:
source: canale_vendita
transform: normalize_channel
mapping:
diretto: direct
booking: ota_booking
expedia: ota_expedia
airbnb: ota_airbnb

guest_language:
source: lingua_preferenza
transform: to_iso639 # it, en, de, fr

guest_country:
source: nazionalita_ospite
transform: to_iso3166 # IT, DE, FR, US

consent_defaults:
consent_marketing: false
consent_ai_analysis: false

END OF DOCUMENT

